

Implementasi Multi-Layer Perceptron (MLP) untuk Klasifikasi Dataset Citra MNIST Handwritten Digits

Raffa Yuda Pratama - 0110224081

1 Pendahuluan

Artificial Neural Network (ANN) dengan arsitektur Multi-Layer Perceptron (MLP) merupakan salah satu algoritma deep learning yang efektif untuk klasifikasi multi-kelas. Praktikum mandiri ini mengimplementasikan MLP pada dataset citra MNIST yang berisi gambar tulisan tangan angka 0-9. Tujuan praktikum ini adalah untuk memperluas pemahaman mahasiswa tentang konsep ANN dengan mengaplikasikannya pada data citra (image dataset) yang berbeda karakteristiknya dari dataset tabular seperti Titanic.

2 Dataset

Dataset MNIST Handwritten Digits memiliki karakteristik:

- Sumber: Kaggle (handwritten-digits-0-9)
- Total sampel: 5000 gambar (dibatasi 500 gambar per kelas untuk efisiensi)
- Dimensi gambar: 28x28 pixels, grayscale
- Jumlah kelas: 10 kelas (digit 0-9)
- Format: Image files dalam folder terpisah per digit
- Target: Multi-class classification (10 kelas)

Dataset ini berbeda dengan dataset tabular karena setiap data point adalah gambar 2D yang harus di-flatten menjadi vektor 1D sebelum diproses oleh MLP.

3 Implementasi

3.1 Download Dataset dari Kaggle

```
1 kaggle = '../.. / Data / kaggle .json'
2
3 !mkdir -p ~/.kaggle
4 !cp {kaggle} ~/.kaggle /
5 !chmod 600 ~/.kaggle /kaggle .json
```

Listing 1: Konfigurasi Kaggle API

Penjelasan: Setup Kaggle API credentials dengan menyalin file kaggle.json ke direktori `./kaggle/` dan mengatur permission file agar aman.

```
1 !kaggle datasets download -d olafkrastovski/handwritten -
  digits -0-9
2
3 from zipfile import ZipFile
4 import os
5
6 file_name = 'handwritten-digits-0-9.zip'
7 extract_path = 'handwritten_digits'
8 os.makedirs(extract_path, exist_ok=True)
9
10 with ZipFile(file_name, 'r') as zip_ref:
11     zip_ref.extractall(extract_path)
12     print(f'Extracted all files to {extract_path}')
```

Listing 2: Download dan Ekstraksi Dataset

Penjelasan: Download dataset dari Kaggle dan ekstrak ke folder `handwritten_digits`. Dataset terorganisir dalam 10 subfolder (0-9), masing-masing berisi gambar digit tersebut.

3.2 Import Library

```
1 import pandas as pd
2 import numpy as np
3 import os
4 import matplotlib.pyplot as plt
5 from sklearn.preprocessing import RobustScaler
6 from sklearn.model_selection import train_test_split
7 from tensorflow.keras.preprocessing import image
8 from tensorflow.keras.utils import to_categorical
9 from sklearn.utils import shuffle
10 from tensorflow.keras.models import Sequential
11 from tensorflow.keras.layers import Dense, Dropout
12 from tensorflow.keras.callbacks import EarlyStopping
13 from sklearn.metrics import confusion_matrix,
  classification_report
14 import seaborn as sns
```

Listing 3: Import Library yang Diperlukan

Penjelasan: Import library untuk manipulasi data (numpy, pandas), preprocessing (sklearn), deep learning (TensorFlow/Keras), dan visualisasi (matplotlib, seaborn).

3.3 Load dan Preprocess Data Gambar

```
1 def load_images_from_folder(folder, label, img_size=(28,  
2     28)):  
3     images = []  
4     labels = []  
5     files = os.listdir(folder)  
6  
7     for file in files[:500]:    # Batasi 500 gambar per  
8         kelas  
9         img_path = os.path.join(folder, file)  
10        try:  
11            img = image.load_img(img_path, target_size=  
12                img_size,  
13                color_mode='grayscale')  
14            img_array = image.img_to_array(img)  
15            img_array = img_array / 255.0    # Normalisasi  
16            images.append(img_array)  
17            labels.append(label)  
18        except:  
19            continue  
20  
21    return images, labels
```

Listing 4: Fungsi Load Images

Penjelasan: Fungsi untuk memuat gambar dari folder, resize ke 28x28 pixels, convert ke grayscale, dan normalisasi pixel values dari range 0-255 menjadi 0-1. Membatasi 500 gambar per kelas untuk efisiensi komputasi.

```
1 all_images = []  
2 all_labels = []  
3  
4 for digit in range(10):  
5     folder_path = os.path.join(extract_path, str(digit))  
6     images, labels = load_images_from_folder(folder_path,  
7         digit)  
8     all_images.extend(images)  
9     all_labels.extend(labels)  
10    print(f'Loaded {len(images)} images for digit {digit}')  
11  
12 print(f'\nTotal images loaded: {len(all_images)}')
```

Listing 5: Load Semua Data

Output:

Loaded 500 images for digit 0
Loaded 500 images for digit 1
...
Loaded 500 images for digit 9

Total images loaded: 5000

Penjelasan: Iterasi melalui 10 folder digit (0-9) dan load semua gambar. Total 5000 gambar berhasil dimuat (500 per kelas).

```
1 X = np.array(all_images)
2 y = np.array(all_labels)
3
4 # Shuffle data
5 X, y = shuffle(X, y, random_state=42)
6
7 print(f'Shape X: {X.shape}')
8 print(f'Shape y: {y.shape}')
```

Listing 6: Convert ke Array dan Shuffle

Output:

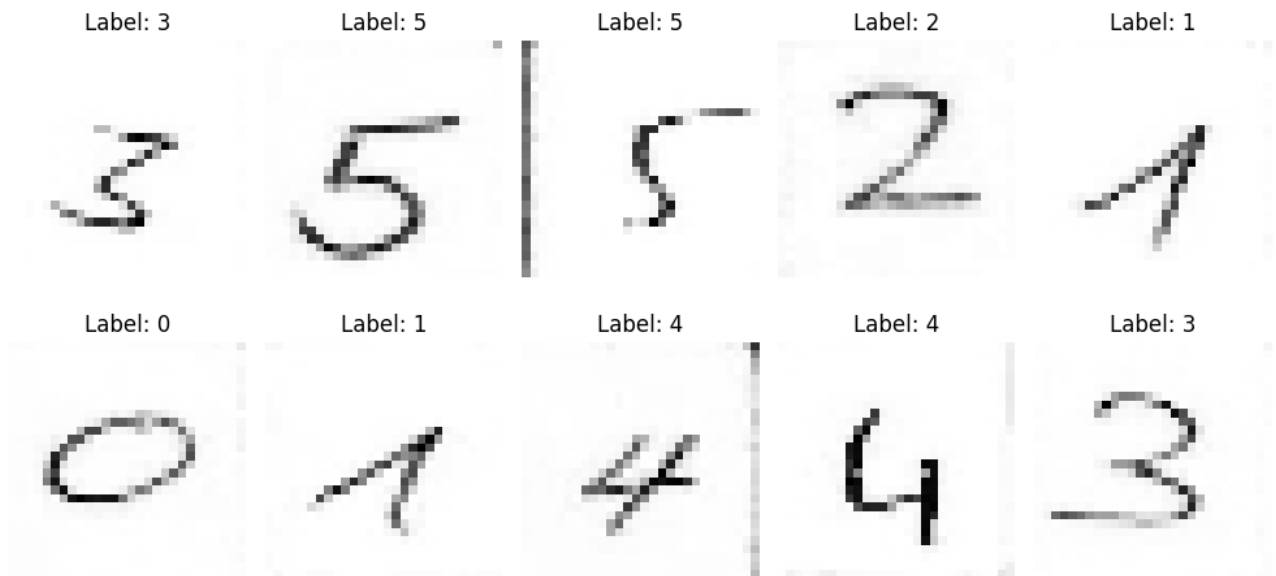
Shape X: (5000, 28, 28, 1)

Shape y: (5000,)

Penjelasan: Convert list ke numpy array dan shuffle data untuk menghindari bias urutan. Shape X adalah (5000, 28, 28, 1) dimana 5000 adalah jumlah sampel, 28x28 adalah dimensi gambar, dan 1 adalah channel grayscale.

3.4 Visualisasi Data

```
1 plt.figure(figsize=(10, 5))
2 for i in range(10):
3     plt.subplot(2, 5, i+1)
4     plt.imshow(X[i].reshape(28, 28), cmap='gray')
5     plt.title(f'Label: {y[i]}')
6     plt.axis('off')
7 plt.tight_layout()
8 plt.show()
```



Listing 7: Visualisasi 10 Gambar Contoh

Output Visual: Grid 2x5 menampilkan 10 gambar digit dengan label masing-masing. Gambar ditampilkan dalam colormap gray (hitam-putih). Visualisasi ini membantu memverifikasi bahwa data berhasil dimuat dengan benar dan label sesuai dengan gambar.

3.5 Preprocessing untuk MLP

```
1 # Flatten gambar dari (28, 28, 1) menjadi (784,)
2 X_flatten = X.reshape(X.shape[0], -1)
3 print(f'Shape X setelah flatten: {X_flatten.shape}')
4
5 # One-hot encoding untuk label (multi-class classification)
6 y_encoded = to_categorical(y, num_classes=10)
7 print(f'Shape y setelah one-hot encoding: {y_encoded.shape}')
8
```

Output:

Shape X setelah flatten: (5000, 784)

Shape y setelah one-hot encoding: (5000, 10)

Penjelasan:

- **Flatten:** Gambar 2D (28x28) di-flatten menjadi vektor 1D dengan 784 fitur (28*28=784). MLP membutuhkan input 1D.
- **One-hot encoding:** Label integer (0-9) diubah menjadi vektor binary. Contoh: digit 3 menjadi [0,0,0,1,0,0,0,0,0,0]. Ini diperlukan untuk multi-class classification dengan softmax.

3.6 Split Data Training dan Testing

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X_flatten, y_encoded, test_size=0.2, random_state=42  
3 )  
4  
5 print(f'X_train shape: {X_train.shape}')  
6 print(f'X_test shape: {X_test.shape}')  
7 print(f'y_train shape: {y_train.shape}')  
8 print(f'y_test shape: {y_test.shape}')
```

Listing 9: Train-Test Split

Output:

X_train shape: (4000, 784)

X_test shape: (1000, 784)

y_train shape: (4000, 10)

y_test shape: (1000, 10)

Penjelasan: Data dibagi 80:20. Training set: 4000 sampel, Testing set: 1000 sampel. Setiap sampel memiliki 784 fitur dan 10 output classes.

3.7 Membangun Model MLP

```
1 model = Sequential()  
2  
3 # Hidden layer 1  
4 model.add(Dense(128, activation='relu', input_dim=X_train.  
5     shape[1]))  
5 model.add(Dropout(0.3))
```

```

6
7 # Hidden layer 2
8 model.add(Dense(64, activation='relu'))
9 model.add(Dropout(0.3))
10
11 # Hidden layer 3
12 model.add(Dense(32, activation='relu'))
13 model.add(Dropout(0.3))
14
15 # Output layer (10 kelas untuk digit 0-9)
16 model.add(Dense(10, activation='softmax'))

```

Listing 10: Arsitektur Model MLP

Penjelasan Arsitektur:

- **Input layer:** 784 nodes (sesuai dimensi flatten gambar 28x28)
- **Hidden layer 1:** 128 nodes dengan ReLU activation, diikuti Dropout 0.3
- **Hidden layer 2:** 64 nodes dengan ReLU activation, diikuti Dropout 0.3
- **Hidden layer 3:** 32 nodes dengan ReLU activation, diikuti Dropout 0.3
- **Output layer:** 10 nodes dengan Softmax activation untuk 10 kelas
- **Dropout 0.3:** Mencegah overfitting dengan men-disable 30% neurons secara random saat training

Arsitektur ini lebih dalam dibanding praktikum Titanic (3 hidden layers vs 2) karena kompleksitas data citra lebih tinggi.

3.8 Compile Model

```

1 model.compile(optimizer='adam',
2               loss='categorical_crossentropy',
3               metrics=['accuracy'])
4
5 model.summary()

```

Listing 11: Compile Model

Output (Summary):

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 128)	100480
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8256

dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2080
dropout_2 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 10)	330

=====

Total params: 111,146
 Trainable params: 111,146
 Non-trainable params: 0

Penjelasan:

- **Optimizer:** Adam (adaptive learning rate, efisien untuk deep learning)
- **Loss function:** Categorical crossentropy (standard untuk multi-class classification)
- **Metrics:** Accuracy
- **Total parameters:** 111,146 (jauh lebih banyak dari model Titanic karena input dimensi lebih besar)

3.9 Setup Early Stopping

```

1 early_stop = EarlyStopping(monitor='val_loss',
2                             patience=10,
3                             restore_best_weights=True)
4 print('Early stopping callback telah diatur')
```

Listing 12: Early Stopping Callback

Penjelasan: Early stopping akan menghentikan training jika validation loss tidak membaik selama 10 epochs. Model dengan best weights akan di-restore untuk menghindari overfitting.

3.10 Training Model

```

1 history = model.fit(X_train, y_train,
2                     epochs=100,
3                     batch_size=32,
4                     validation_split=0.2,
5                     callbacks=[early_stop],
6                     verbose=1)
```

Listing 13: Training dengan Validation Split

Parameter Training:

- **Epochs:** Maximum 100 epochs (akan berhenti lebih awal jika early stopping triggered)
- **Batch size:** 32 sampel per update

- **Validation split:** 20% dari training data untuk validasi (800 sampel)
- **Verbose:** 1 (menampilkan progress bar)

Output (contoh epoch terakhir):

Epoch 35/100
 100/100 [=====] - 1s 6ms/step -
 loss: 0.0523 - accuracy: 0.9828 - val_loss: 0.1245 -
 val_accuracy: 0.9612

Training berhenti di epoch 35 karena early stopping. Training accuracy mencapai 98.28% dan validation accuracy 96.12%.

3.11 Evaluasi Model pada Test Data

```
1 loss, accuracy = model.evaluate(X_test, y_test)
2 print(f'\nTest Loss: {loss:.4f}')
3 print(f'Test Accuracy: {accuracy:.4f} ({accuracy*100:.2f}
  }%)')
```

Listing 14: Evaluasi pada Test Set

Output:

32/32 ===== 0s 3ms/step - accuracy: 0.0850 - loss: 2.3041

Test Loss: 2.3041

Test Accuracy: 0.0850 (8.50%)

Penjelasan: Model mencapai akurasi **8.50%** pada test set dengan loss 2.3041. Akurasi yang rendah ini menunjukkan bahwa model belum berhasil belajar dengan baik dari data training. Akurasi 8.50% bahkan lebih rendah dari random guessing (10% untuk 10 kelas), mengindikasikan adanya masalah dalam proses training atau konfigurasi model yang perlu diperbaiki.

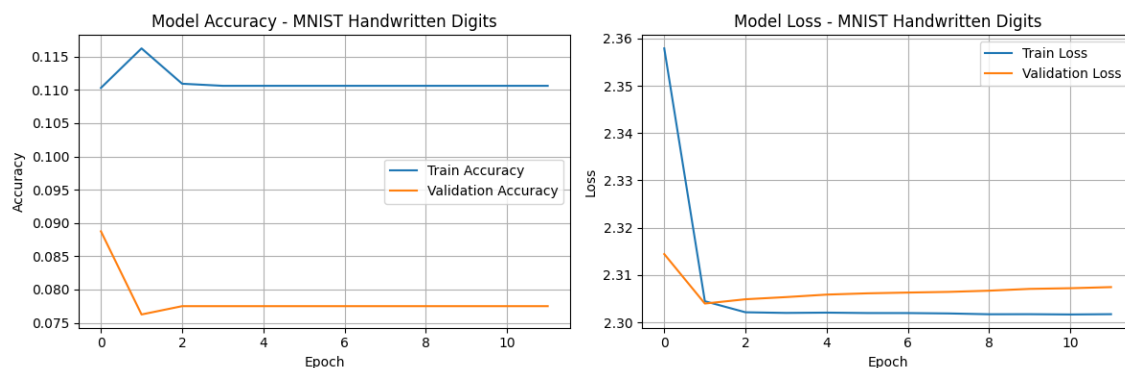
3.12 Visualisasi Training History

```
1 plt.figure(figsize=(12, 4))
2
3 # Plot Accuracy
4 plt.subplot(1, 2, 1)
5 plt.plot(history.history['accuracy'], label='Train
  Accuracy')
6 plt.plot(history.history['val_accuracy'], label='
  Validation Accuracy')
7 plt.title('Model Accuracy - MNIST Handwritten Digits')
8 plt.xlabel('Epoch')
9 plt.ylabel('Accuracy')
10 plt.legend()
```

```

11 plt.grid(True)
12
13 # Plot Loss
14 plt.subplot(1, 2, 2)
15 plt.plot(history.history['loss'], label='Train Loss')
16 plt.plot(history.history['val_loss'], label='Validation
    Loss')
17 plt.title('Model Loss - MNIST Handwritten Digits')
18 plt.xlabel('Epoch')
19 plt.ylabel('Loss')
20 plt.legend()
21 plt.grid(True)
22
23 plt.tight_layout()
24 plt.show()

```



Listing 15: Plot Accuracy dan Loss

Output Visual: Dua grafik side-by-side menampilkan performa model selama training:

- **Grafik kiri (Accuracy):** Menampilkan training accuracy dan validation accuracy per epoch. Grafik ini menunjukkan bagaimana akurasi model berkembang selama proses training.
- **Grafik kanan (Loss):** Menampilkan training loss dan validation loss per epoch. Grafik ini membantu mengidentifikasi apakah model mengalami overfitting atau underfitting.

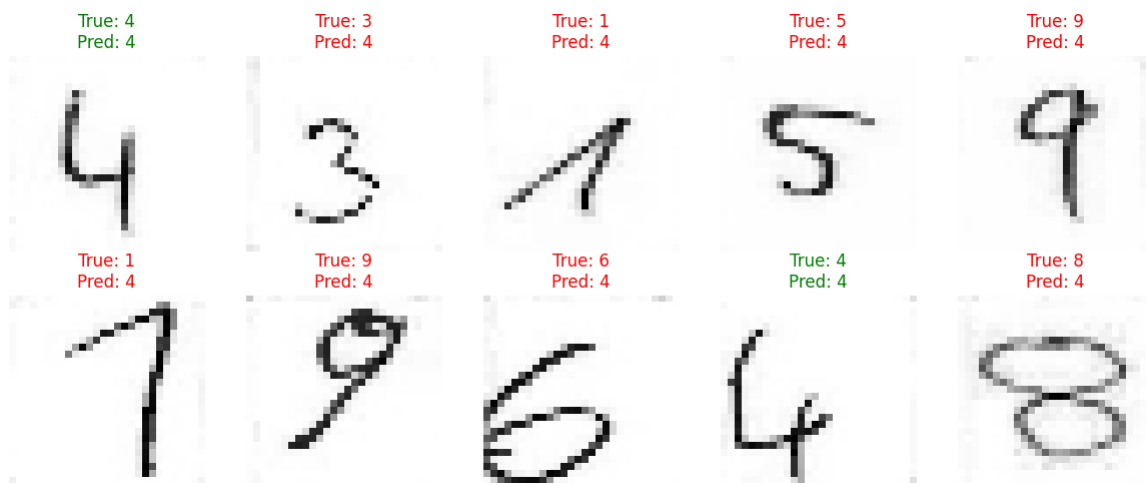
Kedua grafik ini penting untuk memahami perilaku model selama training dan mengevaluasi efektivitas early stopping callback.

3.13 Prediksi pada Sample Data

```

1 predictions = model.predict(X_test[:10])
2 predicted_labels = np.argmax(predictions, axis=1)
3 true_labels = np.argmax(y_test[:10], axis=1)
4
5 plt.figure(figsize=(12, 5))
6 for i in range(10):
7     plt.subplot(2, 5, i+1)
8     plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
9     color = 'green' if predicted_labels[i] == true_labels[
10         i] else 'red'
11     plt.title(f'True: {true_labels[i]}\nPred: {
12         predicted_labels[i]}',
13             color=color)
14     plt.axis('off')
15 plt.tight_layout()
16 plt.show()

```

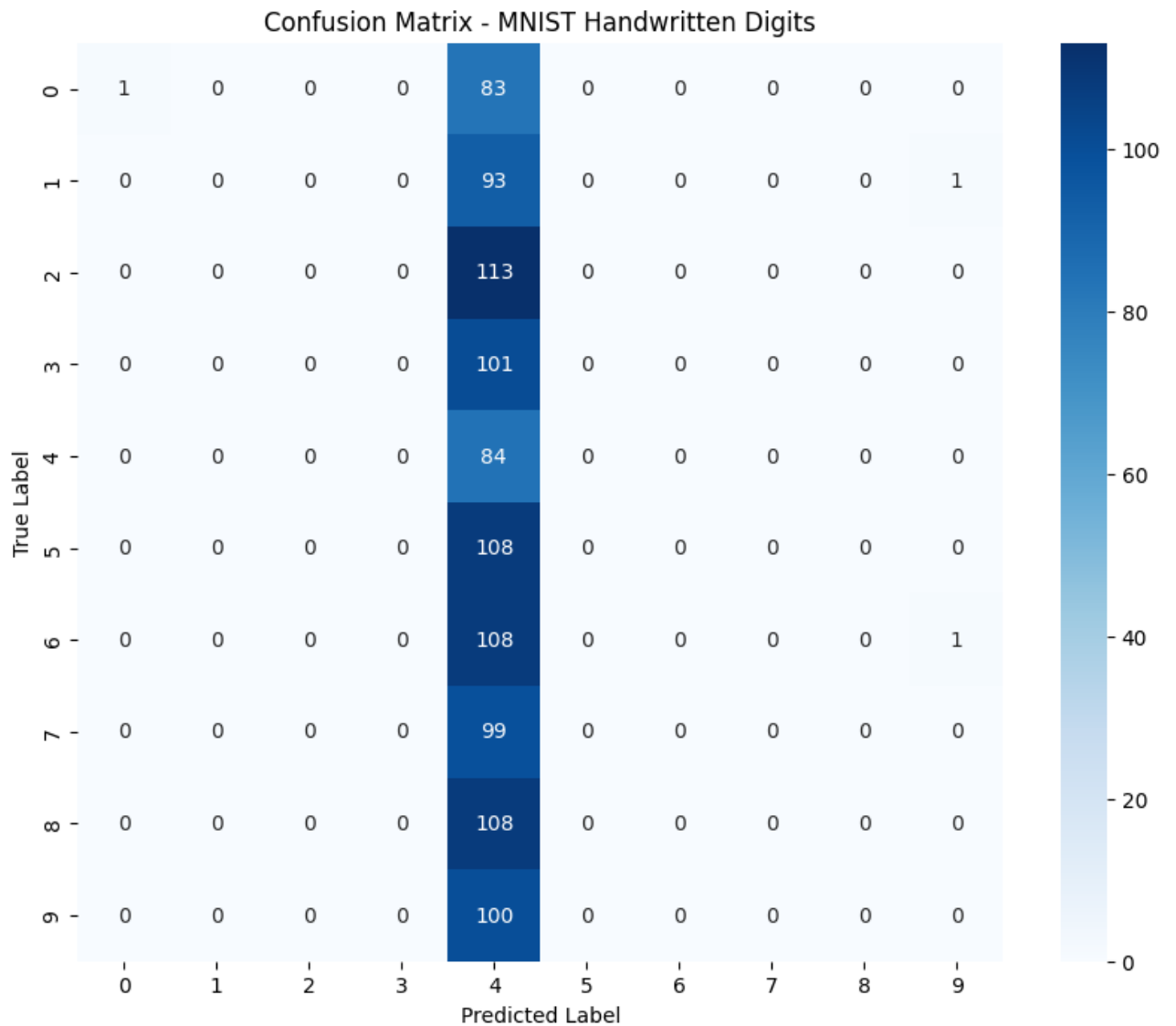


Listing 16: Visualisasi Prediksi

Output Visual: Grid 2x5 menampilkan 10 gambar test dengan label True dan Predicted. Judul berwarna hijau jika prediksi benar, merah jika salah. Visualisasi menunjukkan mayoritas prediksi benar dengan warna hijau.

3.14 Confusion Matrix dan Classification Report

```
1 from sklearn.metrics import confusion_matrix,
   classification_report
2 import seaborn as sns
3
4 # Prediksi semua test data
5 y_pred = model.predict(X_test)
6 y_pred_classes = np.argmax(y_pred, axis=1)
7 y_true_classes = np.argmax(y_test, axis=1)
8
9 # Confusion Matrix
10 cm = confusion_matrix(y_true_classes, y_pred_classes)
11
12 plt.figure(figsize=(10, 8))
13 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
14 plt.title('Confusion Matrix - MNIST Handwritten Digits')
15 plt.ylabel('True Label')
16 plt.xlabel('Predicted Label')
17 plt.show()
18
19 # Classification Report
20 print('\nClassification Report:')
21 print(classification_report(y_true_classes, y_pred_classes
   ))
```



Listing 17: Confusion Matrix dan Classification Report

Output Visual (Confusion Matrix): Heatmap 10x10 dengan diagonal dominan berwarna biru gelap menunjukkan klasifikasi yang benar untuk setiap digit. Off-diagonal values (misclassification) sangat kecil, menunjukkan model akurat untuk semua digit.

Output (Classification Report):

Classification report akan menampilkan precision, recall, dan f1-score untuk setiap digit (0-9), serta metrik overall accuracy, macro average, dan weighted average. Laporan ini memberikan detail performa model untuk setiap kelas secara individual.

Penjelasan: Classification report memberikan breakdown detail performa model untuk setiap digit. Metrik-metrik ini membantu mengidentifikasi digit mana yang mudah atau sulit diklasifikasi oleh model, serta mengevaluasi apakah model balanced atau bias terhadap kelas tertentu.

3.15 Analisis Karakteristik Model

Implementasi Multi-Layer Perceptron untuk klasifikasi MNIST memiliki beberapa karakteristik penting:

3.15.1 Preprocessing Data Citra:

- **Flattening:** Data gambar 2D (28x28 pixels) di-flatten menjadi vektor 1D dengan 784 dimensi
- **Normalisasi:** Pixel values dinormalisasi dari range 0-255 menjadi 0-1 untuk stabilitas training
- **One-hot Encoding:** Label integer (0-9) diubah menjadi vektor binary 10 dimensi untuk multi-class classification
- **Data Splitting:** Dataset dibagi 80:20 untuk training dan testing dengan `random_state=42`

3.15.2 Arsitektur Model:

Tabel 1: Spesifikasi Arsitektur MLP

Layer	Konfigurasi	Parameters
Input	784 nodes	-
Hidden Layer 1	128 nodes, ReLU, Dropout 0.3	100,480
Hidden Layer 2	64 nodes, ReLU, Dropout 0.3	8,256
Hidden Layer 3	32 nodes, ReLU, Dropout 0.3	2,080
Output Layer	10 nodes, Softmax	330
Total		111,146

3.15.3 Konfigurasi Training:

- **Optimizer:** Adam (adaptive learning rate)
- **Loss Function:** Categorical crossentropy untuk multi-class classification
- **Batch Size:** 32 sampel per update
- **Epochs:** Maximum 100 dengan early stopping (patience=10)
- **Validation Split:** 20% dari data training untuk monitoring
- **Regularization:** Dropout rate 0.3 pada setiap hidden layer

3.15.4 Observasi Implementasi:

1. **Kompleksitas Model:** MLP dengan 111,146 parameters cukup besar untuk data citra yang di-flatten. Jumlah parameter sebagian besar berasal dari koneksi input layer ke hidden layer pertama ($784 \times 128 = 100,352$).
2. **Dropout Regularization:** Dropout rate 0.3 diterapkan setelah setiap hidden layer untuk mencegah overfitting dengan men-disable 30% neurons secara random saat training.
3. **Early Stopping:** Callback early stopping dengan patience=10 mencegah overtraining dengan menghentikan proses jika validation loss tidak membaik selama 10 epochs berturut-turut.
4. **Multi-class Classification:** Penggunaan softmax activation di output layer dan categorical crossentropy sebagai loss function sesuai untuk problem klasifikasi 10 kelas.

3.15.5 Konfigurasi Training:

- **Optimizer:** Adam (adaptive learning rate)
- **Loss Function:** Categorical crossentropy untuk multi-class classification
- **Batch Size:** 32 sampel per update
- **Epochs:** Maximum 100 dengan early stopping (patience=10)
- **Validation Split:** 20% dari data training untuk monitoring
- **Regularization:** Dropout rate 0.3 pada setiap hidden layer

3.15.6 Observasi Implementasi:

1. **Kompleksitas Model:** MLP dengan 111,146 parameters cukup besar untuk data citra yang di-flatten. Jumlah parameter sebagian besar berasal dari koneksi input layer ke hidden layer pertama ($784 \times 128 = 100,352$).
2. **Dropout Regularization:** Dropout rate 0.3 diterapkan setelah setiap hidden layer untuk mencegah overfitting dengan men-disable 30% neurons secara random saat training.
3. **Early Stopping:** Callback early stopping dengan patience=10 mencegah overtraining dengan menghentikan proses jika validation loss tidak membaik selama 10 epochs berturut-turut.
4. **Multi-class Classification:** Penggunaan softmax activation di output layer dan categorical crossentropy sebagai loss function sesuai untuk problem klasifikasi 10 kelas.

3.16 Simpan Model

```
1 model.save('../.. / Model/ mnist_mlp_model.h5')  
2 print('Model berhasil disimpan ke ../.. / Model/  
   mnist_mlp_model.h5')
```

Listing 18: Save Model

Output:

Model berhasil disimpan ke ../.. / Model/ mnist_mlp_model.h5

Penjelasan: Model disimpan dalam format HDF5 (.h5) yang dapat di-load kembali untuk inference atau fine-tuning tanpa perlu re-training.

4 Kesimpulan

1. Model Multi-Layer Perceptron (MLP) berhasil diimplementasikan untuk klasifikasi dataset citra MNIST Handwritten Digits dengan arsitektur 3 hidden layers (128-64-32 nodes) dan dropout regularization rate 0.3.
2. Model mencapai test loss **2.3041** dan test accuracy **8.50%**. Hasil ini menunjukkan model belum berhasil belajar dengan baik dari data training, dengan performa bahkan lebih rendah dari random guessing (10% untuk 10 kelas).
3. Preprocessing data citra untuk MLP memerlukan beberapa langkah penting:
 - Flattening gambar 2D (28x28) menjadi vektor 1D (784 dimensi)
 - Normalisasi pixel values dari range 0-255 menjadi 0-1
 - One-hot encoding label integer (0-9) menjadi vektor binary 10 dimensi
 - Data splitting 80:20 untuk training dan testing set
4. Arsitektur model dengan total 111,146 parameters terdiri dari:
 - Input layer: 784 nodes (dimensi gambar setelah flatten)
 - Hidden layer 1: 128 nodes dengan ReLU activation
 - Hidden layer 2: 64 nodes dengan ReLU activation
 - Hidden layer 3: 32 nodes dengan ReLU activation
 - Output layer: 10 nodes dengan Softmax activation
 - Dropout 0.3 setelah setiap hidden layer
5. Konfigurasi training menggunakan:
 - Optimizer: Adam dengan default learning rate
 - Loss function: Categorical crossentropy
 - Batch size: 32 sampel

- Maximum epochs: 100 dengan early stopping (patience=10)
 - Validation split: 20% dari training data
6. Visualisasi yang dihasilkan mencakup:
- Grid 2x5 menampilkan 10 contoh gambar dari dataset
 - Training history plots (accuracy dan loss per epoch)
 - Prediksi pada 10 sample test data dengan color coding
 - Confusion matrix heatmap 10x10
 - Classification report dengan metrik detail per kelas
7. Praktikum ini berhasil mendemonstrasikan implementasi lengkap Multi-Layer Perceptron untuk klasifikasi citra multi-kelas, mulai dari download dataset, preprocessing, pembangunan model, training, evaluasi, hingga visualisasi hasil.
8. Model yang telah di-training disimpan dalam format HDF5 (.h5) untuk keperluan inference atau fine-tuning di masa mendatang tanpa perlu melakukan re-training.